

CDMO Report

Antonio Gravina, antonio.gravina@studio.unibo.it
Maksim Omelchenko, maksim.omelchenko@studio.unibo.it
Matteo Fasulo, matteo.fasulo@studio.unibo.it

July 7, 2024

1 Introduction

This report presents the pertinent concepts and strategies employed to address the Multiple Courier Planning (MCP) problem, which is a variant of the Vehicle Routing Problem (VRP). The MCP problem is a slight variation of the VRP, specifically known as the Capacitated Vehicle Routing Problem (CVRP). Original CVRP has the same capacity for all the vehicles while the analysed problem involves different capacities.

1.1 Input Parameters

All models share the same objective variable with lower and upper bounds. For the CP model the instances remain unaltered, while for SMT and MIP the instances consider the first node as the depot. The following input parameters, which are common across all models, are used:

- Number of couriers: m
- Number of items: n
- Capacities (loads) of the couriers: l
- Sizes of the items: s
- Distance matrix: D

1.2 Objective Variable and Bounds

The lower and upper bounds of the objective variable are computed based on the last row and last column values of the distance matrix D , which represents the depot. Let r_{n+1} , c_{n+1} be the last row and last column of D , and let i_r , i_c be the indices of $\max(r_{n+1})$ and $\max(c_{n+1})$. The lower bound is then defined as follows:

$$\text{Lower bound} = \max(c_{i_r} + \max(r), r_{i_c} + \max(c)) \quad (1)$$

The intuition behind the lower bound is that the minimum possible distance a courier can travel is the one where it delivers one item and then returns to the depot. This implies that the lower bound value is the one where all the couriers deliver just one item before going back to the depot. In this way, we are sure that all the items are delivered, thus the objective function is tightened, and for some instances, the lower bound corresponds to the known optimal solution.

As for the upper bound, let D be the known distance matrix. The maximum value per row of the matrix is computed as an array A . A is then sorted in ascending order, excluding the first $m - 1$ values. The upper bound is given by the sum of the sliced array:

$$\text{Upper bound} = \sum_{i=m}^{n+1} A_i \quad (2)$$

The intuition behind the upper bound is that in the worst-case scenario, there is only one courier delivering $n - m + 1$ items, while the others deliver just one item and then return to the depot. Furthermore, the busiest courier undertakes the most expensive tour.

1.3 Preprocessing

The preprocessing phase merely involves parsing the distance matrix D as a numpy array to verify its symmetry. For the SMT and MIP models, last row and column become first row and column and everything is shifted by one. This is performed in a very small time, thus it was not counted in the effective search time allowed for solving the instances. The key idea is to simplify the implementation of the constraints by considering the depot as the first node in the Graph.

1.4 Project Execution and Challenges

The project took approximately two months during which each team component actively contributed to all the solution approaches. The hardest challenge was the modelling of the SMT and its optimization due to the encoding size. Despite our best efforts, both SMT and MIP models continue to have difficulty in finding solutions for larger instances.

In the process of selecting solvers, our preference leaned towards open-source solvers that could be easily installed in a Linux environment.

Using SMTLIB for SMT proved to be impractical as our encoding involved

three-dimensional decision variables, resulting in large output file sizes (i.e. instance 11 $\rightarrow$ 1.8GB filesize).

To ensure the use of a “solver-independent” language, we opted for PuLP, which not only supports a wide range of LP and MIP solvers but also offers plugins for AMPL and MPS export.

2 Symmetry breaking

There are 2 symmetries found to break:

1. Applied **only** if distance matrix is symmetric. Every path of a courier can be traversed in a reversed order thus giving a symmetric solution. For example paths 0 - 3 - 2 - 0 and 0 - 2 - 3 - 0 are the same when the matrix is symmetric (due to the Graph being undirected). To break that symmetry, the following constraint is added: if $depot, a_1 \dots a_n, depot$ is a path of some courier then $a_1 \leq a_n$.
2. As stated by Coelho and Laporte (2014) [1], the most effective symmetry breaking constraint is the application of a hierarchical order (HC1) in the distribution of items to couriers. This constraint implies that if item i is served by courier k then there exist another item with smaller index served by courier $k - 1$. However this applies only if all couriers have the same capacity. Given that we have an heterogeneous fleet with varying capacities, we have modified the original concept into the following constraint:

$$(l[k1] = l[k2]) \implies \mathcal{M}_{k1,n+1} \leq \mathcal{M}_{k2,n+1} \quad \forall k1 \in 1..m - 1, k2 \in k + 1..m \quad (3)$$

This still imposes an order on the first item delivered for couriers having the same capacity l_k .

The first constraint is stronger if the problem has a lot of couriers with different capacities (as it is related with the path and not with the capacity of couriers), while the second one is stronger if there are multiple couriers having the same capacity.

By trying both the constraints individually and together, we have found that just the second one works better for our problem. The combination of both still works for some instances, but in very large ones it leads to no solution within the time limit. Thus, we chose to use just the second symmetry breaking constraint.

3 CP Model

CP model was implemented in MiniZinc[2] and then solved by Gecode[3] and Chuffed[4] solvers.

3.1 Decision variables

The CP model has the following decision variables:

- $\text{obj_dist} \in \mathbb{N}^{1 \times m}$ with domain $[0.. \text{Upper bound}]$ holding per each entry the total tour cost for the courier.
- $\text{couriers_nodes} \in \mathbb{N}^{m \times n+1}$ with domain $[1..n+1]$ where $\text{couriers_nodes}[k, i] = j$ means that j is the successor of i and $\text{couriers_nodes}[k, i] = i$ means that i is not in the tour for the courier $k \in 1..m$. For simplicity, couriers_nodes will be called $\mathcal{M}$. Additional details about the interpretation of $\mathcal{M}$ are available in the Appendix (section 6).
- $\text{obj} \in \mathbb{N}$ with domain $[\text{Lower bound}.. \text{Upper bound}]$ representing the objective value.
- $\text{loads} \in \mathbb{N}^{1 \times n}$ with domain $[1..m]$ representing which item is assigned to which courier.
- $\text{couriers_loads} \in \mathbb{N}^{1 \times m}$ with domain $[\min(s).. \max(l) + 1]$ holding the total load carried by each courier.

3.2 Objective function

The objective function is:

$$\max \left(\sum_{i=1}^{n+1} D_{i, \mathcal{M}_{k,i}} \right), \forall k \in 1..m \text{ with } \mathcal{M}_{k,i} \neq i \quad (4)$$

which is then minimized.

The minimization of the objective function involves finding the minimum cost of the most expensive tour (in terms of distances). The bounds for the objective function have been already described in section 1.2.

3.3 Constraints

The following constraints have been implemented in the CP model.

Each courier leaves the depot, and depot appears once

By adding this constraint:

$$\forall i \in 1..n, \forall k \in 1..m : \text{count}(\mathcal{M}_{k,i}, n+1, 1) \wedge \mathcal{M}_{k,n+1} \neq n+1 \quad (5)$$

we make sure that the depot appears exactly once in each row of $\mathcal{M}$. Moreover, we enforce that the last element of every row in $\mathcal{M}$ must not be the depot. This means that the node representing the depot cannot point to itself (i.e. the depot must be reachable by every courier).

Each item must be delivered by one courier

We impose this via the global constraint:

$$\forall i \in 1..n : \text{count}(\mathcal{M}_{*,i}, i, m - 1) \quad (6)$$

which enforces the number of occurrences of the item i in $\mathcal{M}$ to be $m - 1$. In fact, if an item i is in self-loop in a certain row k , it means that the courier k does not deliver that item. So, saying that the item i is in self-loop for $m - 1$ couriers is equal to saying that the item i must be delivered just by one courier.

Each courier must not exceed its load capacity

We first initialize an array *loads* which represents the assignments of the items to the couriers:

$$\text{loads}[i] = k \iff \mathcal{M}_{k,i} \neq i, \forall i \in 1..n \quad (7)$$

It means that item i is assigned to the courier $\text{loads}[i]$. To force that the couriers do not exceed their maximum capacity, the bin packing capacity was enforced among the l , *loads* and s arrays:

$$\text{bin_packing_capa}(l, \text{loads}, s) \quad (8)$$

which ensures that each item i with weight $s[i]$, be put into $\text{loads}[i]$ such that the sum of the weights of the items in each bin b does not exceed the capacity $l[b]$.

Subcircuit constraint

Since the problem requires each courier to perform a subtour and do not visit twice the same node, the subcircuit global constraint was enforced:

$$\forall k \in 1..m : \text{subcircuit}(\mathcal{M}_{k,*}) \quad (9)$$

which constrains the nodes in each row to define a subcircuit. In particular, $\mathcal{M}_{k,i} = j$ means that j is the successor of i and $\mathcal{M}_{k,i} = i$ means that i is not in the circuit for each courier $k \in 1..m$.

Total path cost per courier

The total path cost for each courier is enforced via the following constraint:

$$\forall k \in 1..m : \text{obj_dist}[k] = \sum_{i=1}^{n+1} D_{i, \mathcal{M}_{k,i}} \quad (10)$$

which means that the total path cost for each courier is the sum of the distances of the visited nodes in his path. In case i is equal to $\mathcal{M}_{k,i}$ then we know for sure that $D_{i,i} = 0$ by definition.

Total load per courier

The total load per each courier is imposed via the following constraint:

$$\forall k \in 1..m : \text{couriers_loads}[k] = \sum_{i=1}^n s_i[\text{loads}[i] = k] \quad (11)$$

which means that the total load for each courier is the sum of the size of the items carried by that courier.

3.4 Validation

Experimental design

All the results are reproducible using the seed 42 during search.

Gecode

Gecode search strategy is different among the instances. Since the first 10 instances are less computationally expensive, the search annotations for the solver did not introduce any randomness in the variables selection nor in the domain value assignment. The search annotations for Gecode use a sequential search of integer searches on $\mathcal{M}$ and loads array. For both the integer searches, first fail variable selection and minimum domain value assignment were used. Starting from instance 11 the search strategy introduces restarts and Large Neighborhood Search (LNS) respectively with Luby sequence[5] and relax and reconstruct annotations of MiniZinc. These two techniques ensures a larger exploration of the search space with LNS being very useful in finding solutions in CVRP problems [6]. Since restarts are useful if there is some randomness in either variable selection or domain assignment, indomain random was employed to make sure that each restart explores new parts of the search space.

Chuffed

In relation to the Chuffed solver, a distinct search strategy was employed due to its lack of support for all the search annotations of the Gecode solver. Similar to Gecode, the initial 10 instances with Chuffed implemented a sequential search on $\mathcal{M}$ and loads array, using first fail variable selection and minimum domain value assignment. Nevertheless, in the majority of the computationally demanding instances, the search continued to use the first fail variable selection heuristic on both $\mathcal{M}$ and loads array. The Luby sequence with restarts did not yield any notable improvement, and LNS was not compatible with the Chuffed solver. The inverse path symmetry breaking constraint (referenced as 3) was not implemented here due to the limited search annotations of Chuffed and the overall search being too time-consuming. Consequently, with both the symmetry breaking constraints, it was unable to find any solution within the time constraint.

Experimental results

CP solver results				
Instance	Gecode	Gecode + SB	Chuffed	Chuffed + SB
1	14	14	14	14
2	226	226	226	226
3	12	12	12	12
4	220	220	220	220
5	206	206	206	206
6	322	322	322	322
7	167	167	167	167
8	186	186	186	186
9	436	436	436	436
10	244	244	244	244
11	742	692	1298	1298
12	423	380	1066	1055
13	754	752	1390	1362
14	1106	1022	1926	1926
15	924	859	1175	1175
16	286	286	356	358
17	1245	1251	1414	1446
18	927	906	1462	1475
19	334	343	944	944
20	1233	1224	1685	1685
21	836	735	1640	1520

Table 1: Results using Gecode and Chuffed with and without symmetry breaking using the search strategy

4 SMT Model

The Satisfiability Modulo Theory (SMT) is an approach to solve optimization problems using different theories [7]. The Python **z3** API is used to solve the problem with SMT. The solution is designed to minimize the use of non-Boolean logic, employing integer logic only for defining capacity constraints and the objective function

4.1 Decision variables

1. 3D boolean variable $TENSOR[i][j][k]$ of size $(n+1) \times (n+1) \times m$ is used to describe the predicate "Courier k traversed arc (i, j) in his optimal tour". For simplicity we will report $TENSOR[i][j][k]$ as X_{ijk} .
2. Array of integers u of length $n+1$ is used as supporting variables to eliminate subtours. Each variable u_i is in the interval $s[i] \leq u_i \leq Q$, where $Q = \max_{k=1}^m(l(k))$.

4.2 Objective function

Defining objective function is straightforward:

$$\max_{k=1}^m \left(\sum_{i=0}^n \sum_{j=0}^n X_{ijk} \cdot D_{ij} \right) \quad (12)$$

Minimize the most expensive path among all the couriers.

4.3 Constraints

The constraints are following:

Every courier leaves the node it enters

$$\bigwedge_{i=0}^n \bigwedge_{j=0}^n \bigwedge_{k=1}^m \left(X_{ijk} \implies \bigvee_{h=0}^n X_{jhk} \right) \quad (13)$$

Each node except the depot is entered and left just once by any courier

$$\bigwedge_{j=1}^n ExactlyOne(X_{*j*}) \quad (14)$$

$$\bigwedge_{j=1}^n ExactlyOne(X_{j**}) \quad (15)$$

Every courier goes through the depot once

$$\bigwedge_{k=1}^m \text{ExactlyOne}(X_{0,*,k}) \quad (16)$$

Forbidding staying in the same node

$$\bigwedge_{i=0}^{n+1} \bigwedge_{k=1}^m \neg X_{iik} \quad (17)$$

With conditions 13, 14, and 15, it is ensured that each node, except the depot, is visited exactly once by the same courier. Condition 16 ensures that every courier goes through the depot once. When combined with 17, results in each courier's path being a set of cycles that can intersect only in the depot. Additionally to that, every set of cycles includes one that passes through the depot.

Eliminating subtours with Miller-Tucker-Zemlin

$$\bigwedge_{i=1}^{n+1} \bigwedge_{j=1}^{n+1} \bigwedge_{k=1}^m (i \neq j) \rightarrow (u_j - u_i \geq s[j] - Q(1 - X_{ijk})) \quad (18)$$

Where $Q = \max(l)$. Having that, all the paths of couriers now are cycles which go through the depot and every node is visited.

Capacity constraints

$$\bigwedge_{k=1}^m \left(\sum_{i=0}^{n+1} \sum_{j=1}^{n+1} s_j \cdot X_{ijk} \right) \leq l_k \quad (19)$$

With condition 19 each courier will respect its maximum capacity.

Symmetry breaking: impose order on first delivery for couriers

$$\bigwedge_{k=0}^{m-2} \bigwedge_{i=1}^n \bigwedge_{j=i+1}^{n+1} (X_{0,j,k} \implies \neg X_{0,i,k+1}) \quad (20)$$

4.4 Validation

Experimental design

SMT model considers the array s (size of elements) and u_i to have length $n + 1$ with $s[0] = 0$ and $u[0] = 0$. This was just a simplification for the implementation due to the mismatch of the sizes while iterating over all nodes in $n + 1$.

The encoding of *exactly one* constraints is performed using the Heule encoding which requires $\frac{(n-3)}{2}$ new variables with $O(n)$ clauses.

All the results are reproducible using the seed 42 during search.

z3

The z3 solver has a built-in method for handling optimization problems. However this method only reduces the upper bound to obtain a optimal solution. In order to achieve a better optimization we can either lower the upper bound when a new solution with a better objective value is found, or adjust both the upper and lower bounds. The algorithm resembles a binary search, decreasing the upper bound when a better solution is identified and increasing the lower bound for UNSAT solutions. Implementing this is straightforward as z3 provides the capability of creating backtrackable points, enabling the model to push and pop to previous states. The pseudocode for the binary search is available in the Appendix section 1.

Experimental results

Instances not reported have unknown solution.

SMT solver results		
Instance	Z3	Z3 + SB
1	14	14
2	226	226
3	12	12
4	220	220
5	206	206
6	322	322
7	211	211
8	186	186
9	436	436
10	244	244
13	1886	1886

Table 2: Results using Z3 with and without symmetry breaking using the binary search strategy.

5 MIP Model

The Mixed Integer Programming (MIP) model has been implemented using the Python library, PuLP [8]. This library is an LP modeler that generates MIP or LP files and has the capability to invoke various solvers for model optimization. For the MIP model, we experimented with several encodings.

5.1 Decision variables

The MIP model shares the same decision variables of the SMT one. The only difference is the name of the objective variable:

- $\text{max_var} \in \mathbb{N}$ with bounds $[\text{lower_bound}..\text{upper_bound}]$ representing the objective function value mentioned in section 1.2

We also use X_{ijk} instead of $TENSOR[i][j][k]$ for simplicity of the paper.

5.2 Objective function

$$\forall k \in 1..m : \left(\sum_{i=0}^n \sum_{j=0}^n X_{ijk} \cdot D_{ij} \right) \leq \text{max_var} \quad (21)$$

5.3 Constraints

Each node except the depot is entered just once

$$\forall j \in 1..n : \sum_{i=0}^n \sum_{k=1}^m X_{ijk} = 1 \quad (22)$$

Every vehicle leaves the depot once

$$\forall k \in 1..m : \sum_{j=0}^n X_{0,j,k} = 1 \quad (23)$$

Vehicle leaves node it enters

$$\forall i \in 0..n, \forall k \in 1..m : \sum_{j=0}^n X_{ijk} - \sum_{j=0}^n X_{jik} = 0 \quad (24)$$

Remove self-loops

$$\forall i \in 0..n, \forall k \in 1..m : X_{iik} = 0 \quad (25)$$

Each vehicle does not exceed its capacity

$$\forall k \in 1..m : \left(\sum_{i=0}^n \sum_{j=1}^n X_{ijk} \cdot s[j] \right) \leq l[k] \quad (26)$$

Sub-tour elimination using Miller-Tucker-Zemlin formulation

[9]

$$\forall k \in 1..m, \forall i \in 1..n, \forall j \in 1..n : u[i] + s[j] \cdot X_{ijk} - Q \cdot (1 - X_{ijk}) \leq u[j] \quad (27)$$

Symmetry breaking constraint: impose ordering on first item delivered

$$\begin{aligned} & \forall k_1 \in 1..m-1, \forall k_2 \in k_1+1..m : \\ & (l[k_1] = l[k_2]) \implies \forall i \in 0..n, \forall j \in i+1..n : \\ & \quad X_{0,j,k_1} + X_{0,i,k_2} \leq 1 \end{aligned} \tag{28}$$

If two couriers share the same capacity then we can impose an order in their first delivered items meaning that both the $(0, j)$ arc for courier k_1 and $(0, i)$ arc for courier k_2 cannot be true simultaneously with $j \leq i$ and $k_1 \leq k_2$. Having the same capacity implies that the two couriers can exchange their tours thus imposing an ordering breaks symmetric solutions.

5.4 Validation

Experimental design

The model evaluation was carried out utilizing the CBC and GLPK solvers. We chose not to use Gurobi due to the requirement of a license for executing larger instances. Although CPLEX was considered, its installation proved to be challenging in a dockerized environment. Additionally, the CBC solver demonstrated some irregularities in the outcomes when a timeout happened. Certain solutions were found to violate the constraints (for instance, having multiple depots in a courier path). Therefore, the solutions derived from the CBC are retained only if the achieved objective value is equal to the highest tour cost across all couriers. All the results are reproducible using the seed 42 during search.

Experimental results

MIP solvers results				
Instance	CBC	CBC + SB	GLPK	GLPK + SB
1	14	14	14	14
2	226	226	226	226
3	12	12	12	12
4	220	220	220	220
5	206	206	206	206
6	322	322	322	322
7	167	167	167	167
8	186	186	186	186
9	436	436	436	436
10	244	244	244	244
13	N/A	718	N/A	N/A

Table 3: Results using CBC and GLPK with and without symmetry breaking

6 Conclusions

- With the CP approach we are able to solve all the instances. In particular, Gecode can solve 12 instances to the optimality, while Chuffed reaches optimality for 10 instances. This difference is probably given by the Large Neighborhood Search strategy used by Gecode.
- SMT is able to solve just the first 10 instances, plus the instance 13. Among those, only the 13th is not solved to the optimality. The difference of performances between CP and SMT is probably caused by the encoding used for the latter, which is not efficient enough to make us solve the big instances. In fact, the solver is not even able to compile the big instances within the time limit.
- MIP solves the same instances of SMT. However, the 13th instance is solved only with the CBC + SB configuration. Even in this case, the main cause is probably the inefficiency of the encoding method.
- Most of the times, for all the strategies and solvers used, the SB configuration has better performances than the non-SB counterpart. Indeed, the objective values we get are often smaller, as well as the time required to find the optimal solution.
- For each model used, the checker *solution.py* did not return errors.

References

- [1] Maryam Darvish, Leandro C Coelho, and Raf Jans. *Comparison of symmetry breaking and input ordering techniques for routing problems*. CIRRELT Québec, QC, Canada, 2020.
- [2] Nicholas Nethercote, Peter J. Stuckey, Ralph Becket, Sebastian Brand, Gregory J. Duck, and Guido Tack. Minizinc: Towards a standard cp modelling language. In Christian Bessière, editor, *Principles and Practice of Constraint Programming – CP 2007*, pages 529–543, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg.
- [3] Gecode Team. Gecode: Generic constraint development environment, 2006. Available from <https://www.gecode.org>.
- [4] G. Chu, Stuckey, P.J., Schutt, A., Ehlers, T., Graeme Gange, and K.F. Chuffed, a lazy clause generation solver, 2024. Available from <https://github.com/chuffed/chuffed>.
- [5] Michael Luby, Alistair Sinclair, and David Zuckerman. Optimal speedup of las vegas algorithms. In *Proceedings of the twenty-fifth annual ACM symposium on Theory of computing*, pages 128–133. ACM, 1993.
- [6] Paul Shaw. Using constraint programming and local search methods to solve vehicle routing problems. In Michael Maher and Jean-Francois Puget, editors, *Principles and Practice of Constraint Programming — CP98*, pages 417–431, Berlin, Heidelberg, 1998. Springer Berlin Heidelberg.
- [7] Clark Barrett and Cesare Tinelli. *Satisfiability Modulo Theories*, pages 305–343. Springer International Publishing, Cham, 2018.
- [8] Iain Dunning, Stuart Mitchell, and Michael O’Sullivan. Pulp: A linear programming toolkit for python, 2011. Available at <https://optimization-online.org/?p=11731>.
- [9] Clair E Miller, Albert W Tucker, and Richard A Zemlin. Integer programming formulation of traveling salesman problems. *Journal of the ACM (JACM)*, 7(4):326–329, 1960.

Appendix

Couriers nodes representation

Each row in the matrix can be conceptually thought as a linked list, where each node points to the next node in the tour for a specific courier. In this context, the “next” node is not determined by a pointer like in a traditional linked list, but by the sequence of the nodes in the row. The value at each position in the row indicates the node visited at that step of the tour, and the next value in the row represents the next node in the tour. Given the following representation of couriers and nodes:

$$\mathcal{M} = \begin{bmatrix} 1 & 8 & 3 & 2 & 4 & 6 & 7 & 5 \\ 1 & 2 & 7 & 4 & 5 & 6 & 8 & 3 \\ 6 & 2 & 3 & 4 & 5 & 8 & 7 & 1 \end{bmatrix}$$

It means that, for the first courier, the tour will be the following:

$$(8) \rightarrow 5 \rightarrow 4 \rightarrow 2 \rightarrow (8)$$

For the second courier, the tour will be:

$$(8) \rightarrow 3 \rightarrow 7 \rightarrow (8)$$

Last courier will undertake the tour:

$$(8) \rightarrow 1 \rightarrow 6 \rightarrow (8)$$

Let's take into account the first courier: initially, we need to check the depot's position (8th) in the array, which will be the first node to visit. Then we jump to the index of the value found at 8th position, which is 5. The tour ends whenever we find again the depot's node.

Z3 Binary Search Algorithm

Algorithm 1 Binary Search for Optimal Solution

Require: s (solver), u (upper bound), l (lower bound), t (timeout)

Ensure: [SOMETHING]

```

1:  $previous \leftarrow True$ 
2: while SAT and time <  $t$  do           ▷ While SAT and not exceeding timeout
3:    $time \leftarrow get\_time()$ 
4:    $d \leftarrow (u - l)/2$                  ▷ Compute the delta to subtract
5:   if  $u == l$  then
6:     break
7:   else
8:      $midpoint \leftarrow u - d$            ▷ Subtract the delta from upper bound
9:   end if
10:   $s.add(obj \leq midpoint)$              ▷ Add the new constraints
11:   $s.add(obj \geq lower\_bnd)$ 
12:  if  $previous$  then
13:     $s.push()$                          ▷ Create backtracking point
14:     $previous \leftarrow False$ 
15:  end if
16:  if UNSAT then
17:     $l \leftarrow midpoint$ 
18:     $previous \leftarrow True$ 
19:     $s.pop()$                            ▷ Backtrack to last point
20:  else if SAT then
21:     $obj \leftarrow objective\_value$ 
22:     $u \leftarrow obj$                    ▷ Update with objective value
23:     $previous \leftarrow False$ 
24:  end if
25: end while

```
